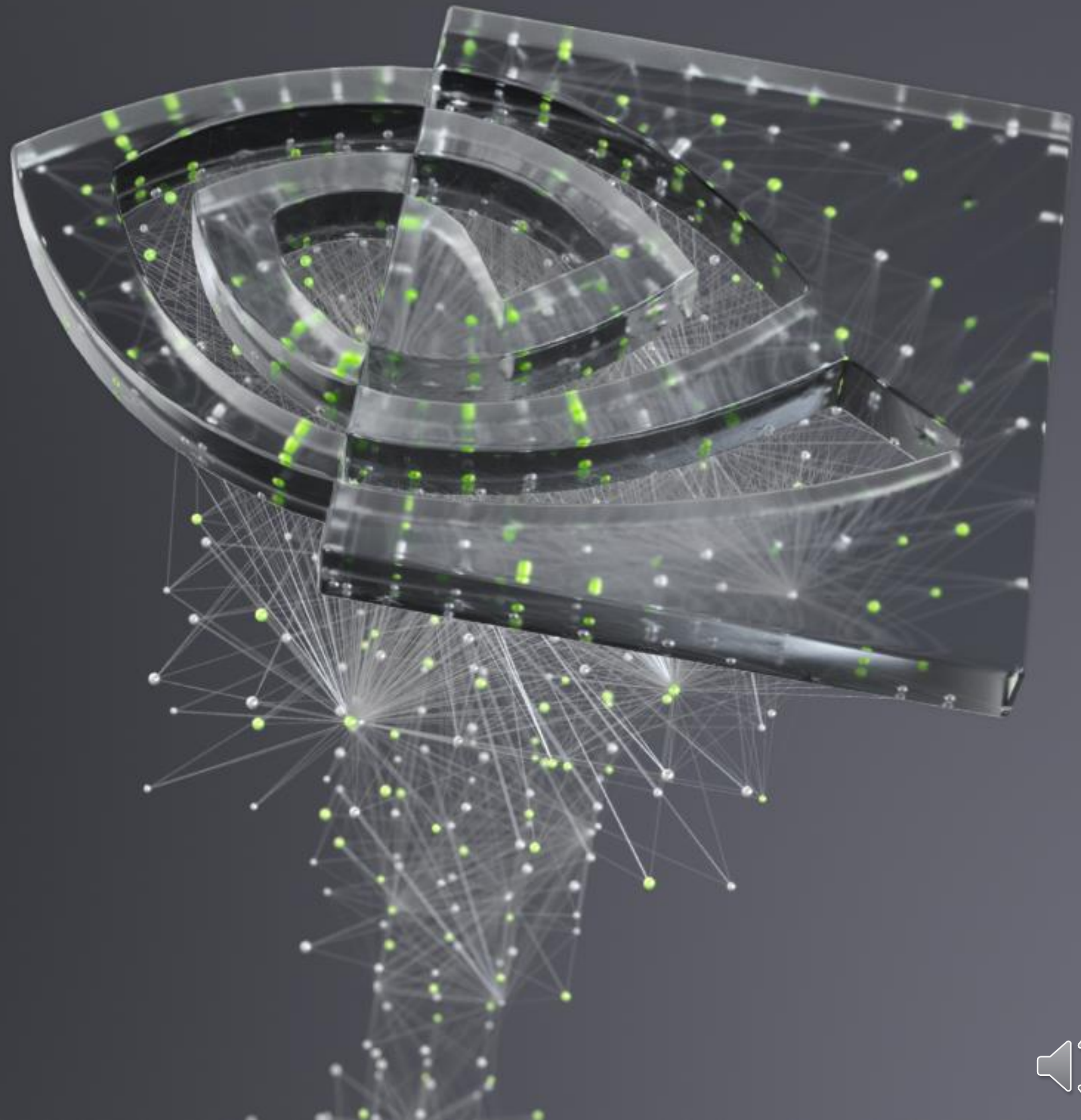




EFFORTLESS CUDA GRAPHS

Rob Van der Wijngaart, April 14, 2021



WE WILL BE STARTING MOMENTARILY

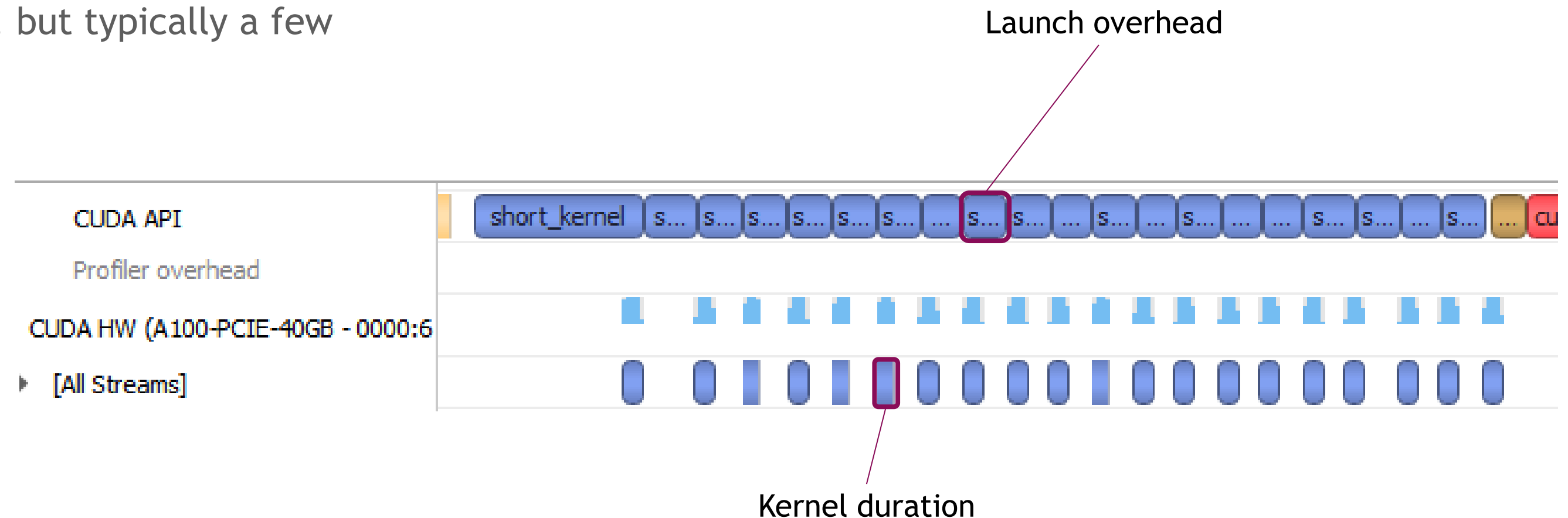
Thank you for joining my talk on Effortless CUDA graphs. I'd like to remind you that I am available for chat during the talk. Please open the "Chat" window on the bottom right of this browser, then click on "Ask the Presenter/Moderator" button to submit your question.



DOCTOR, MY KERNELS HURT

What should I do if my workload has many small kernels

- ▶ What you cannot see can still hurt you
- ▶ Workloads often contain many small kernels—find out using NsightSystems
- ▶ What is small? It's relative, but typically a few microseconds or less
- ▶ Problem: kernel launch incurs several microseconds overhead on CPU
- ▶ Solution: CUDA Graphs!



DOCTOR, MY KERNELS HURT

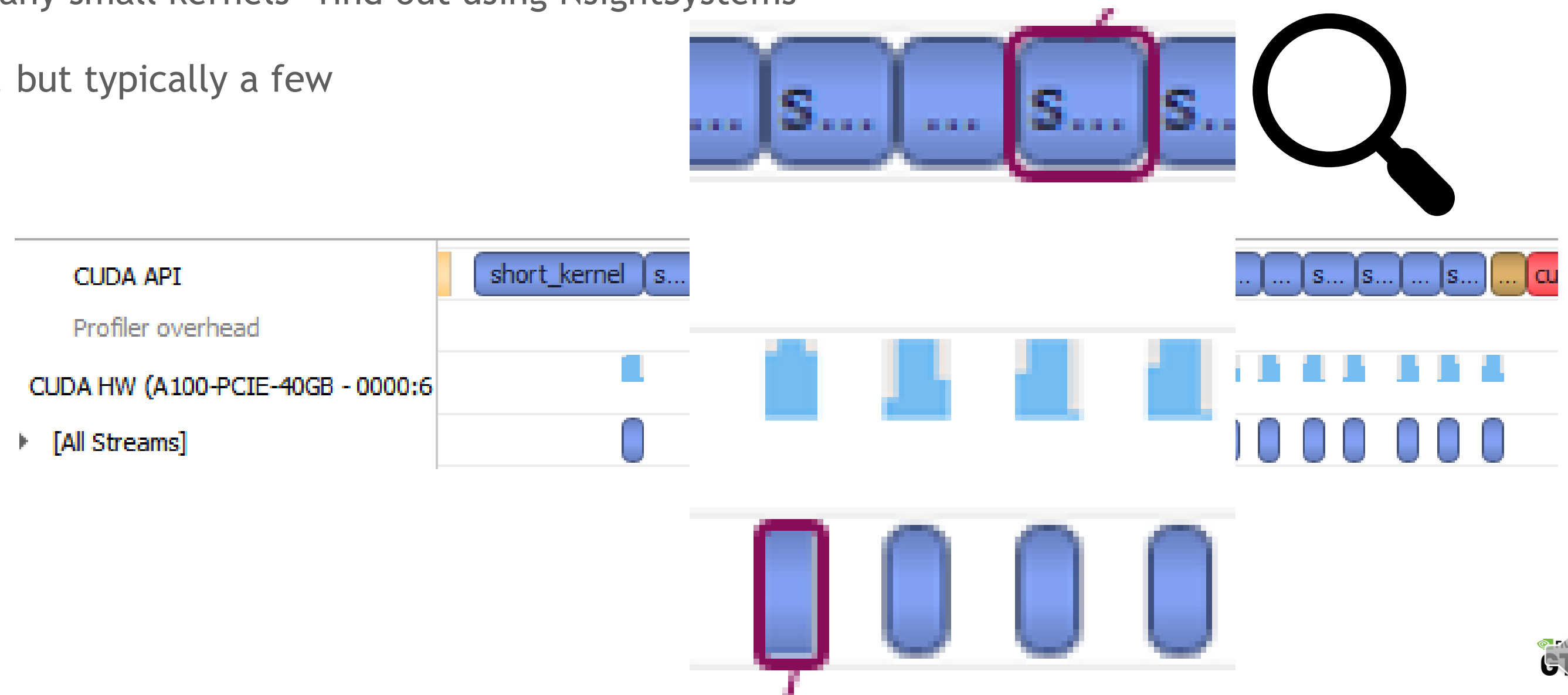
What should I do if my workload has many small kernels

- ▶ What you cannot see can still hurt you
- ▶ Workloads often contain many small kernels—find out using NsightSystems

- ▶ What is small? It's relative, but typically a few microseconds or less

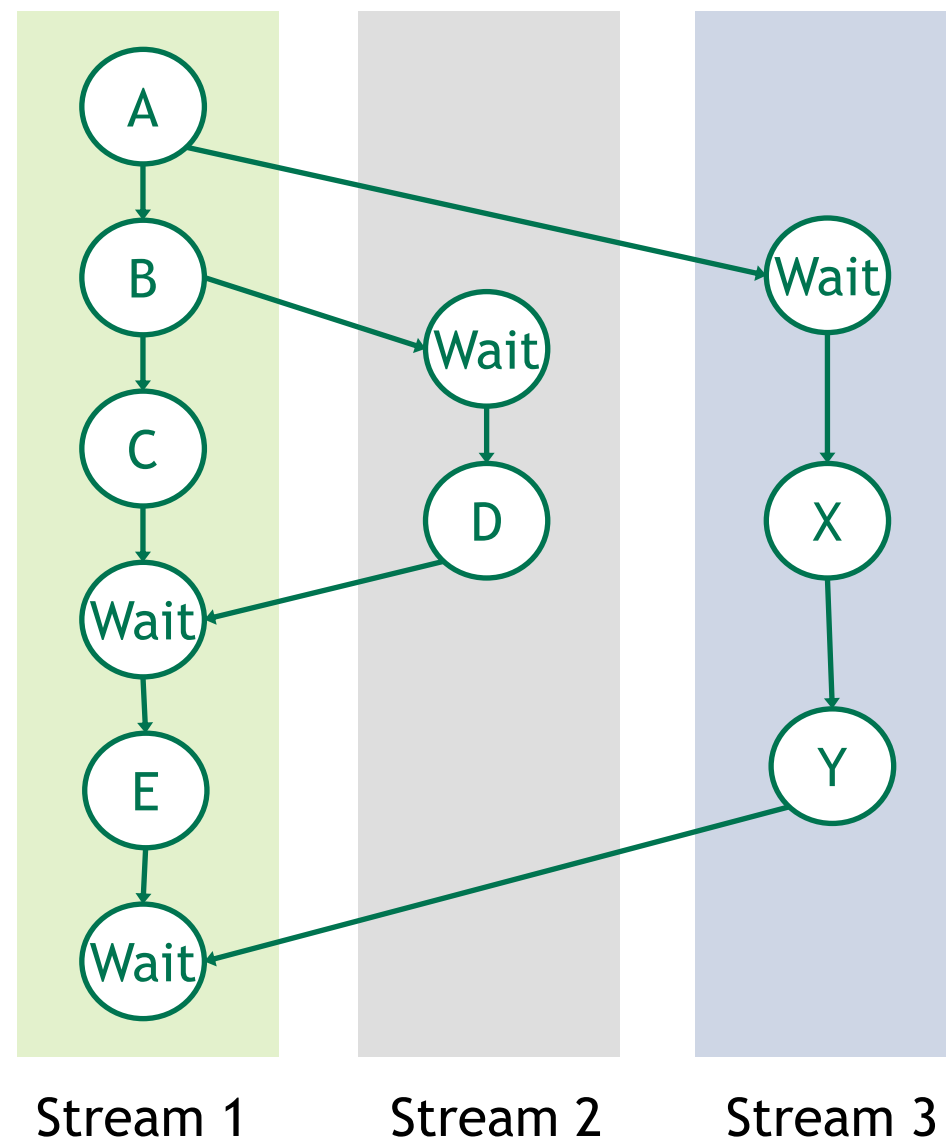
- ▶ Problem: kernel launch incurs several microseconds overhead on CPU

- ▶ Solution: CUDA Graphs!



ALL CUDA WORK FORMS A GRAPH

CUDA Work in Streams

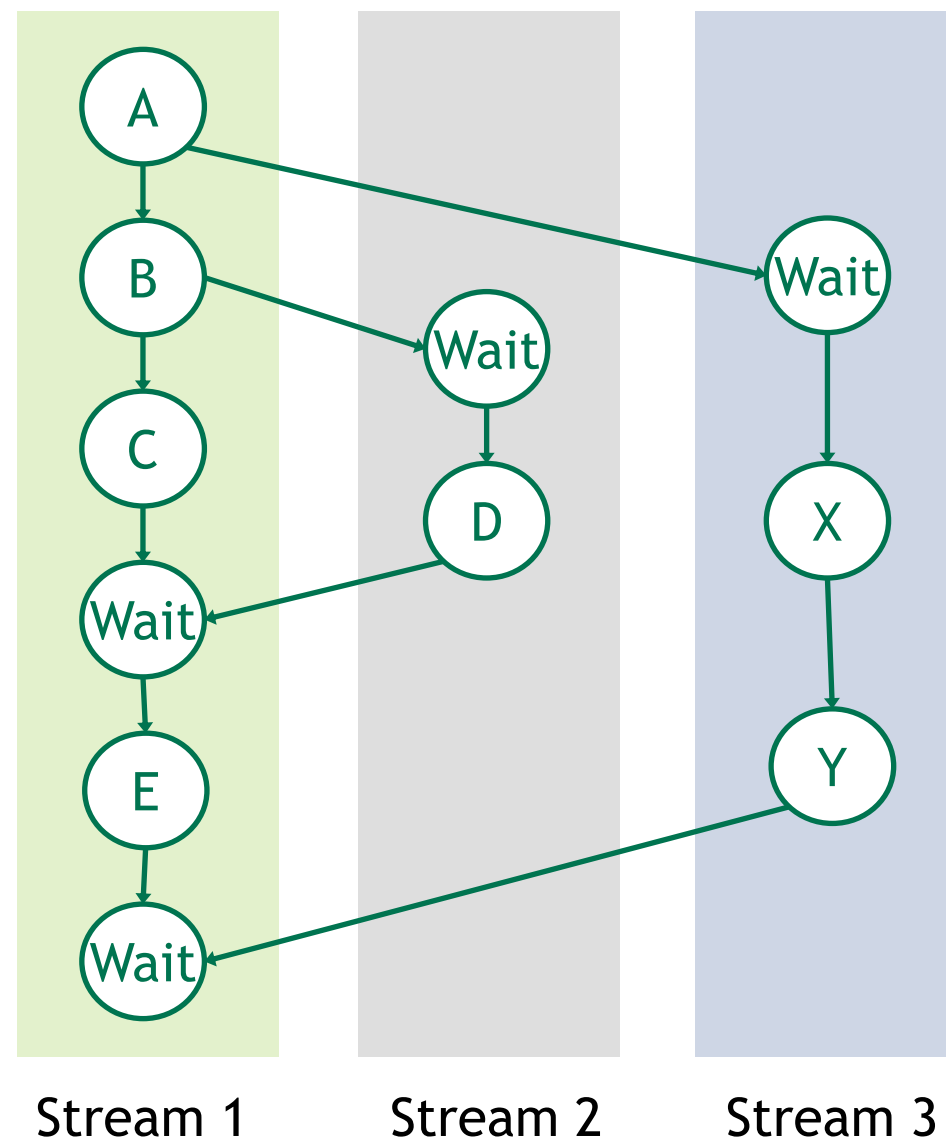


ALL CUDA WORK FORMS A GRAPH

Graph node: any asynchronous CUDA operation

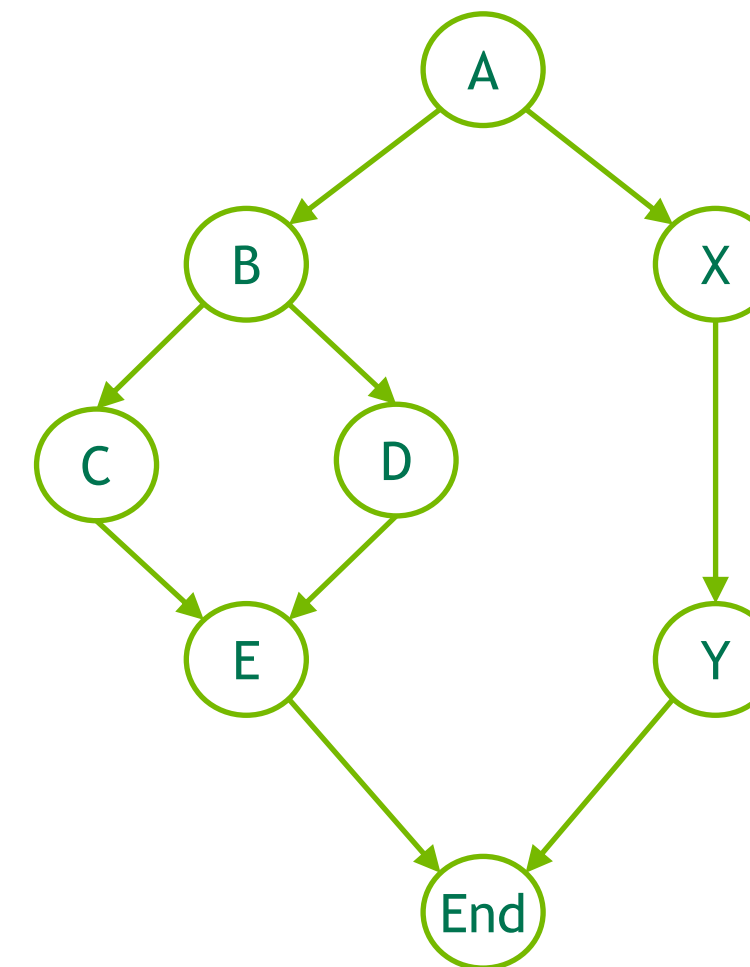
Graph edge: implied or explicit synchronization between nodes

CUDA Work in Streams



Any CUDA
stream
workflow can
be mapped to
a graph

Graph of Dependencies



DEFINITION OF A CUDA GRAPH

Graph node: any asynchronous CUDA operation

Graph edge: implied or explicit synchronization between nodes

Set of operations, connected by dependencies

Operation is (mostly) one of:

Kernel Launch

CUDA kernel running on GPU

CPU Function Call

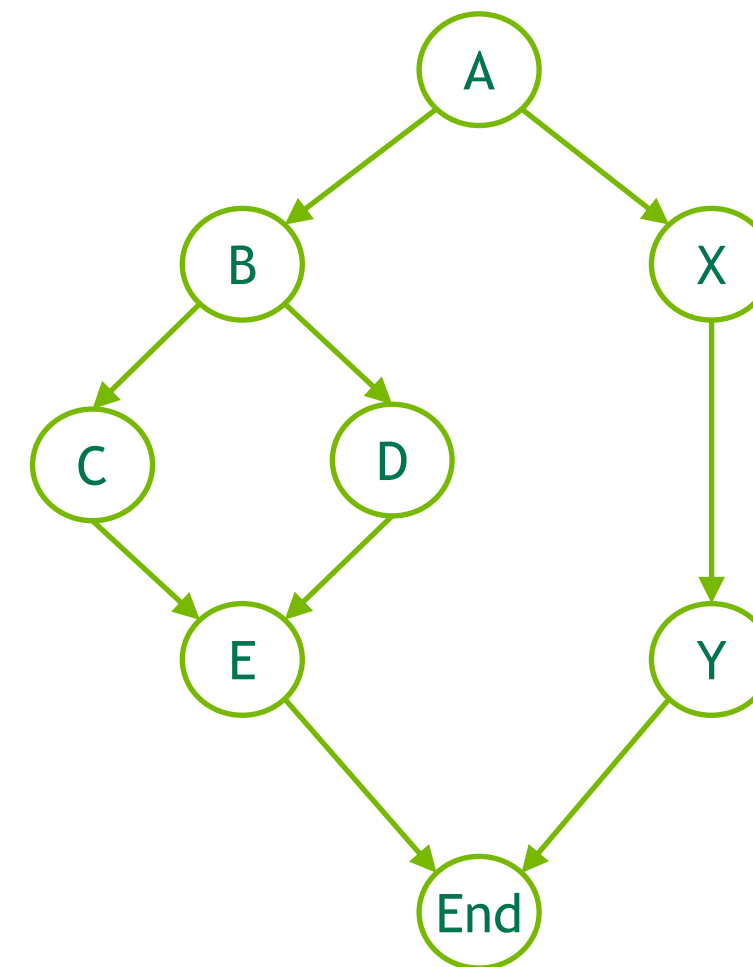
Callback function on CPU

cudaMemcpy/MemsetAsync

GPU data management

Sub-Graph

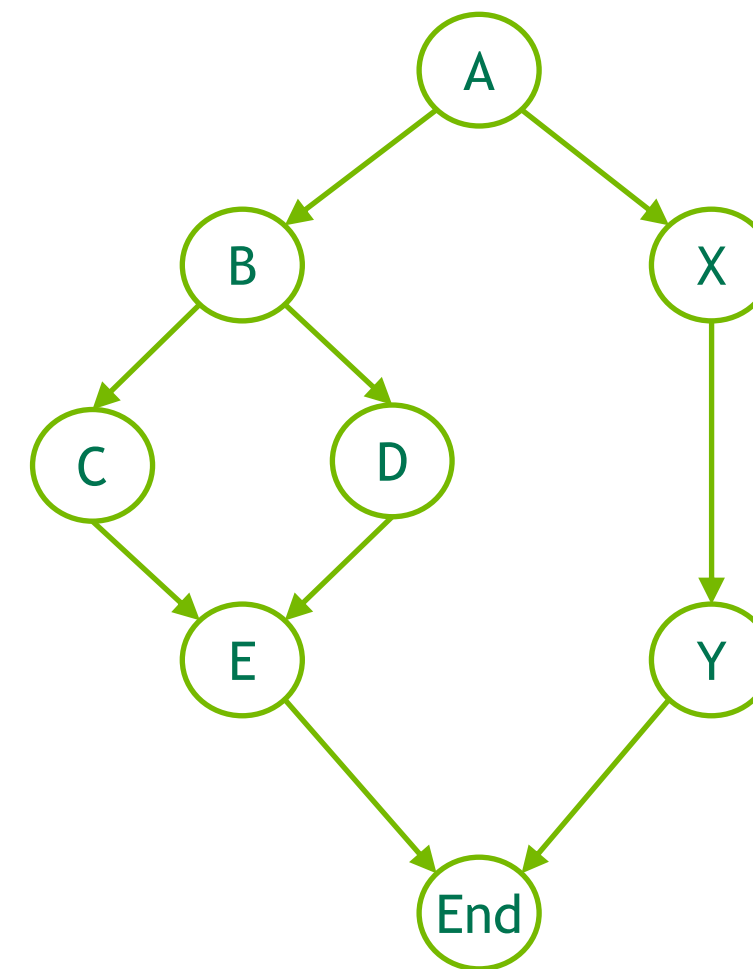
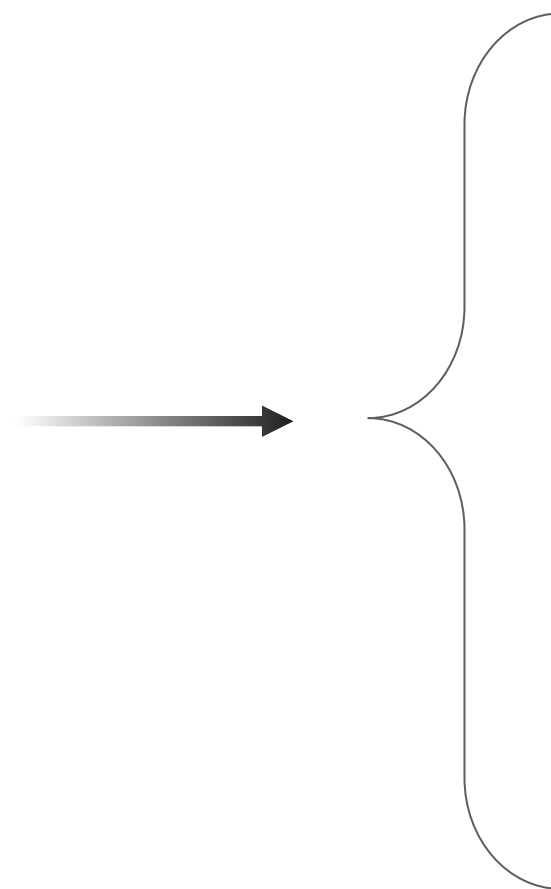
Graphs are hierarchical



CUDA GRAPH EXECUTION MODEL

Graphs generated once, launched repeatedly

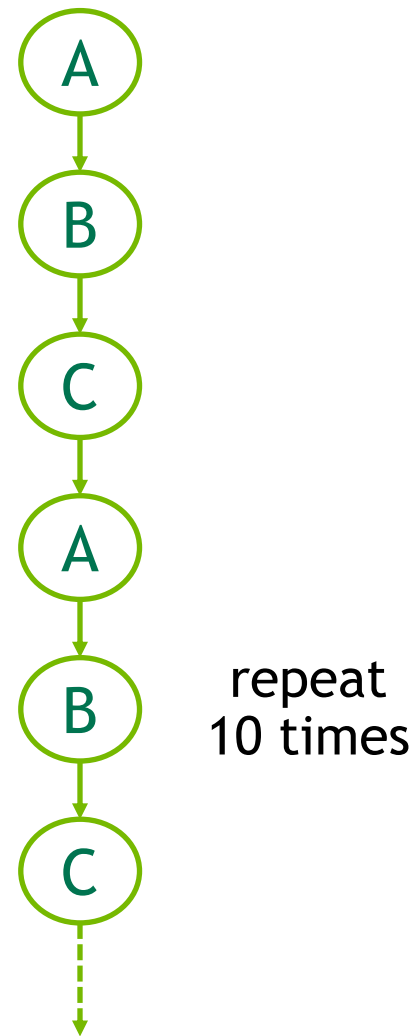
```
for(int i=0; i<1000; i++)  
{  
    launch_graph( G );  
}
```



CUDA GRAPH PROGRAMMING MODEL

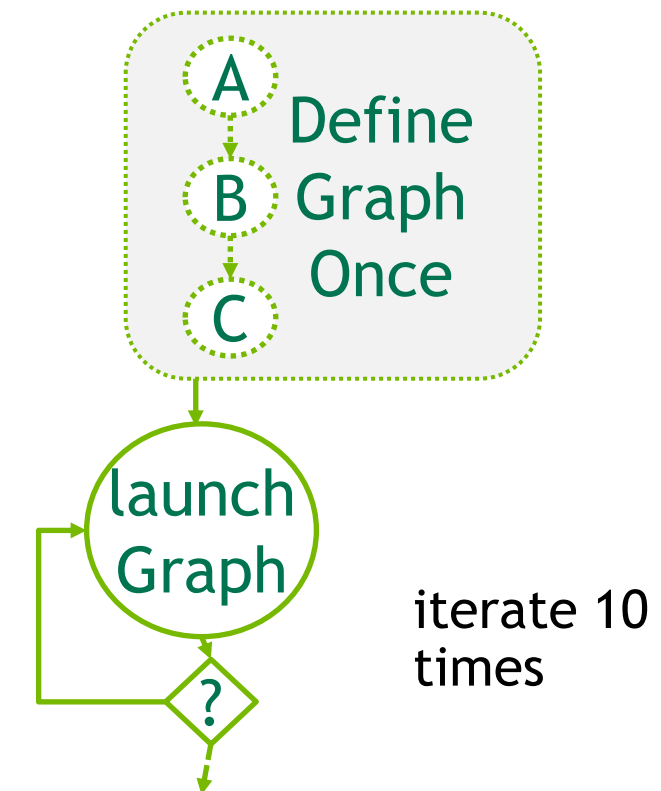
Kernel and Graph execution restrictions

Stream Launch



Parameters: **may** change
Topology: **may** change

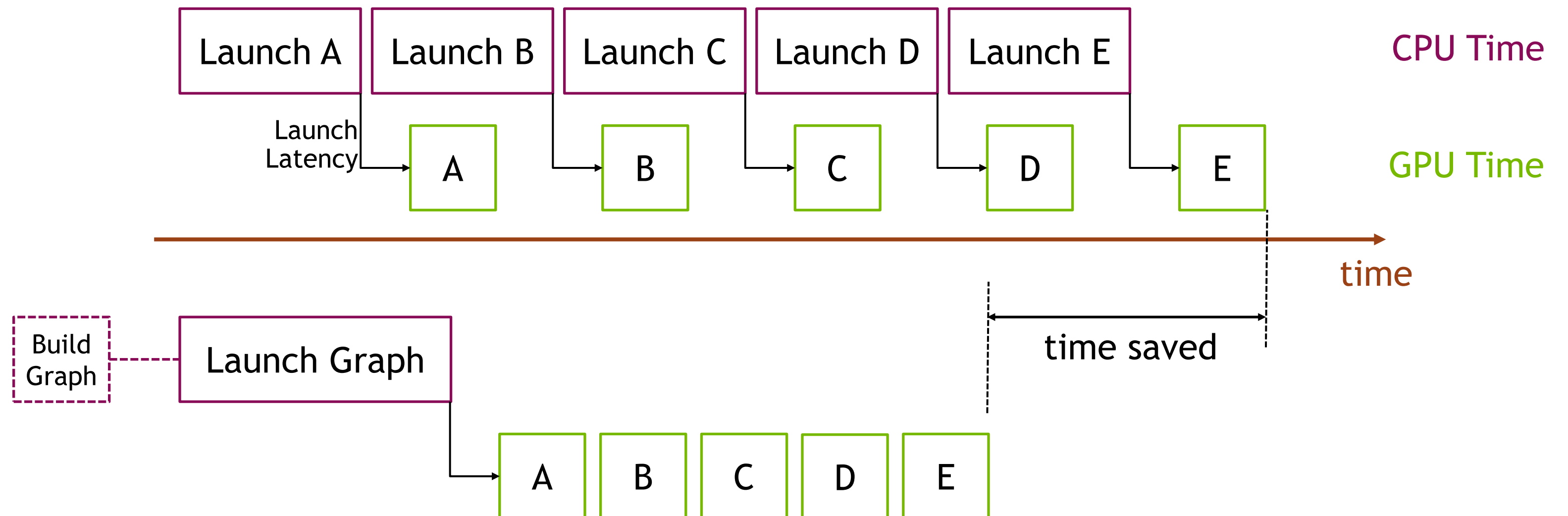
Graph Relaunch



Parameters: **may not** change
Topology: **may not** change

LAUNCH OVERHEAD REDUCTION

Graph launch submits all work at once, reducing CPU cost



When kernel runtime is short, execution time dominated by CPU launch cost

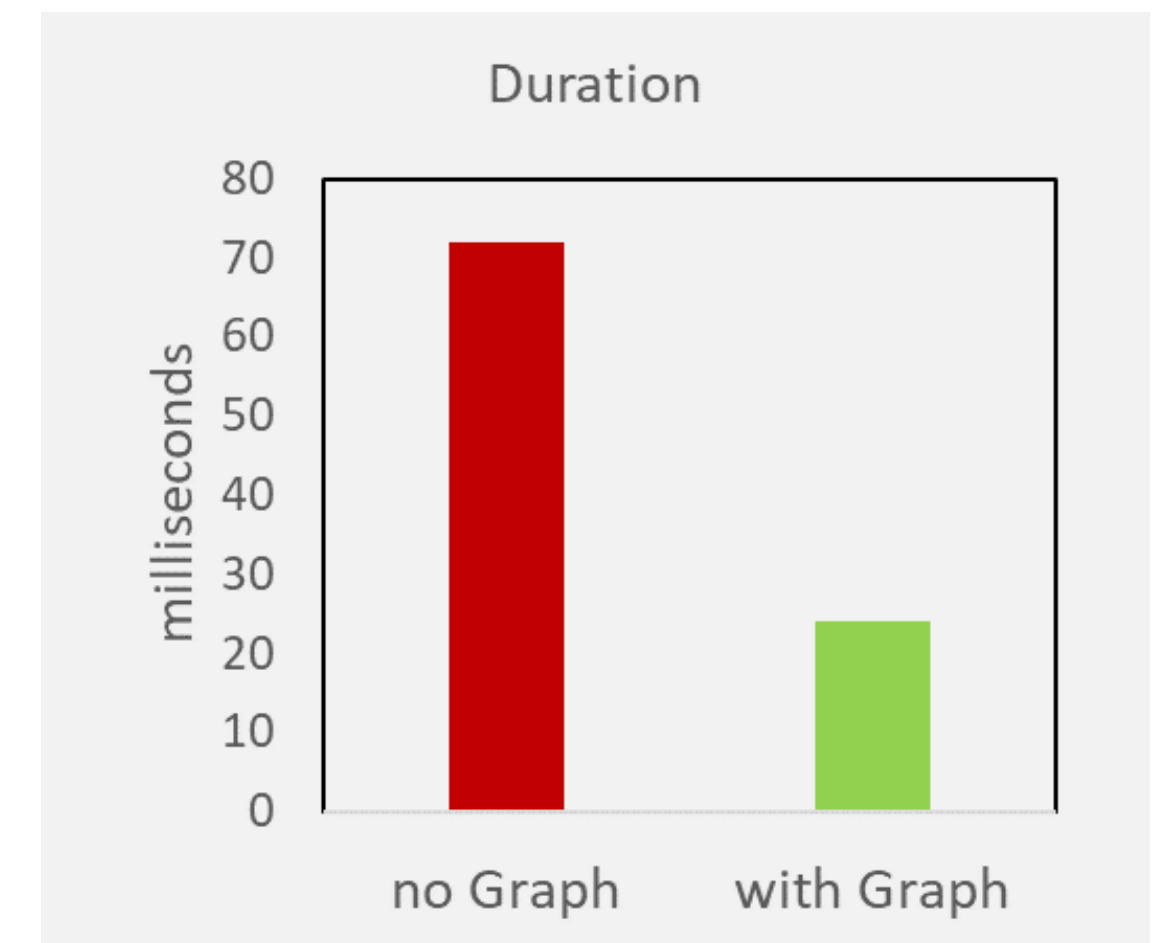
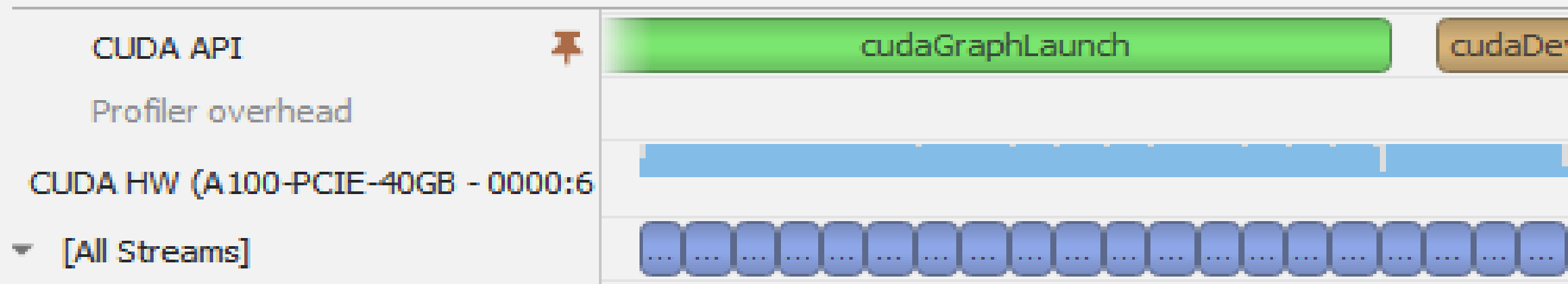
INCREASE THROUGHPUT USING GRAPHS

Kernels themselves also speed up a bit inside Graphs

no Graph



with Graph

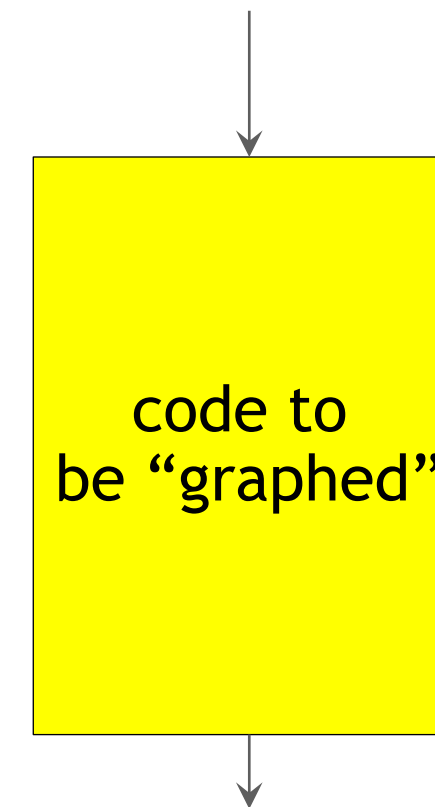


WAYS TO CREATE CUDA GRAPHS

Explicit construction

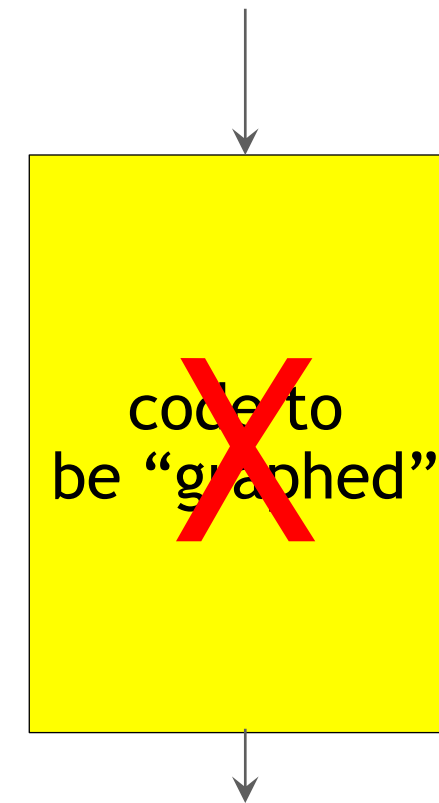
WAYS TO CREATE CUDA GRAPHS

Explicit construction



WAYS TO CREATE CUDA GRAPHS

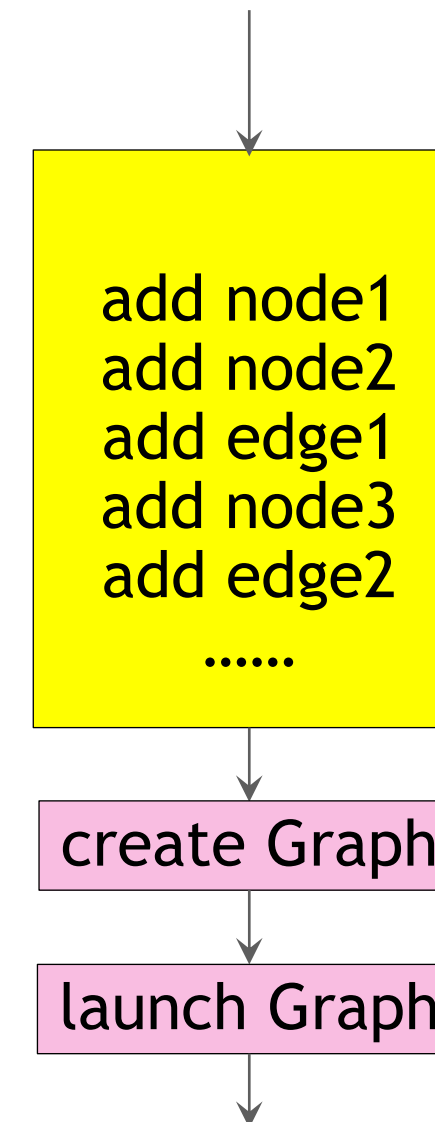
Explicit construction



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

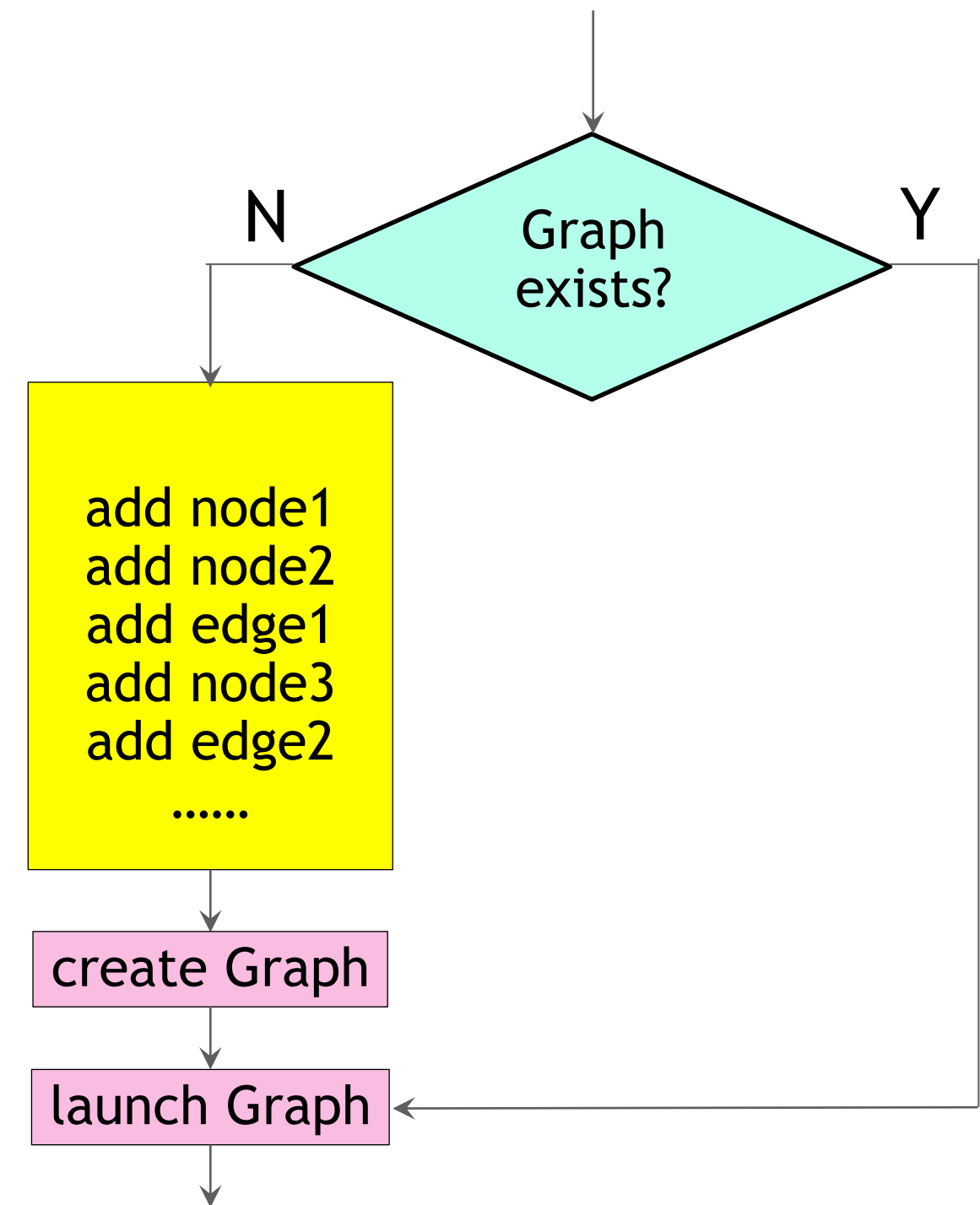
- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

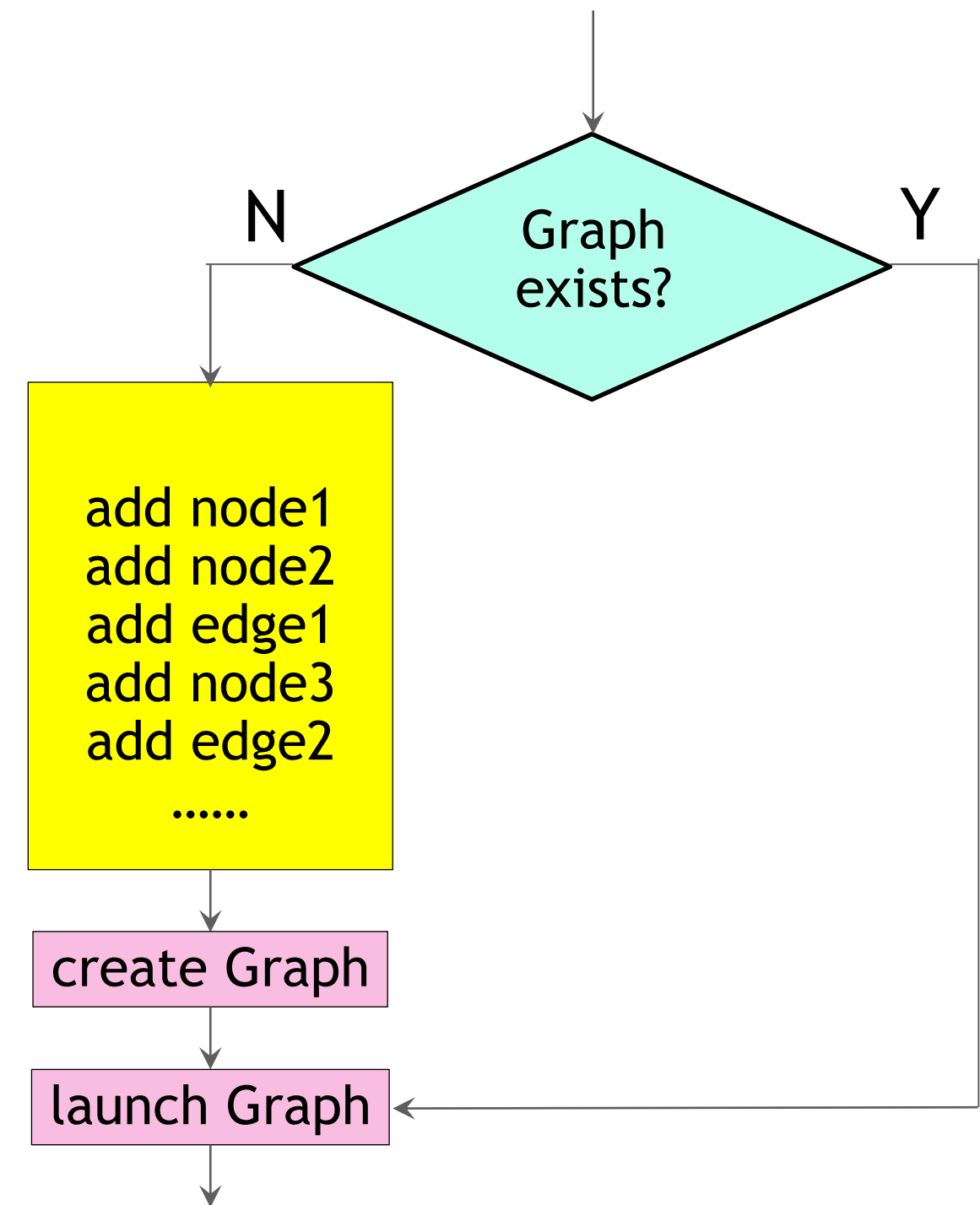
- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

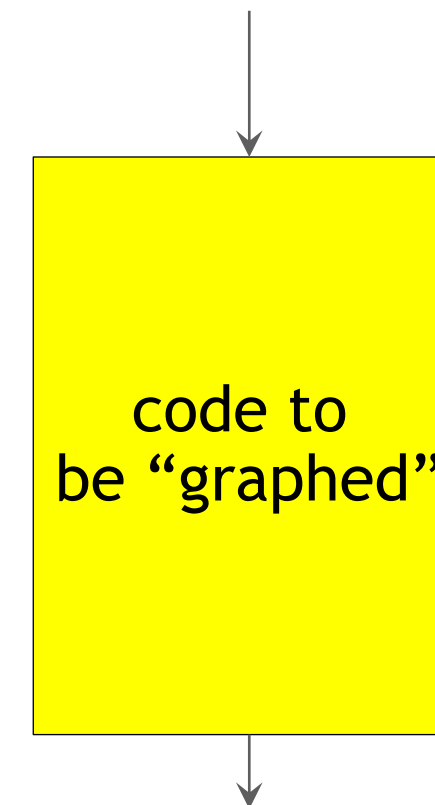


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

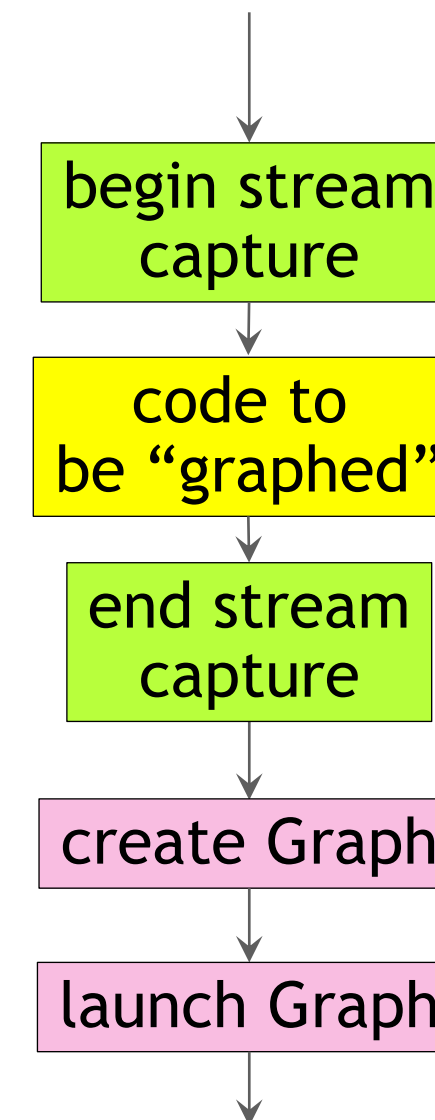


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

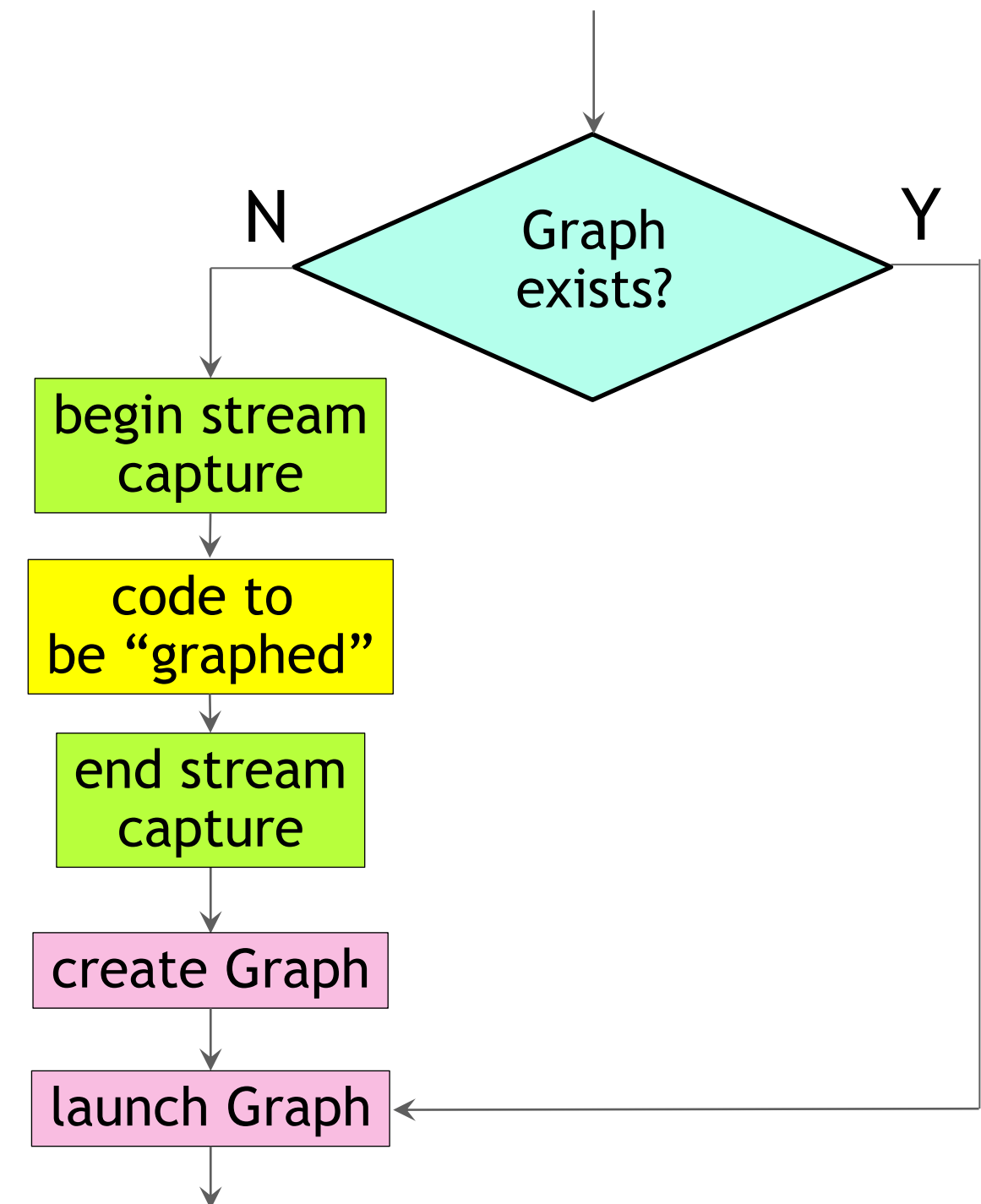


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls



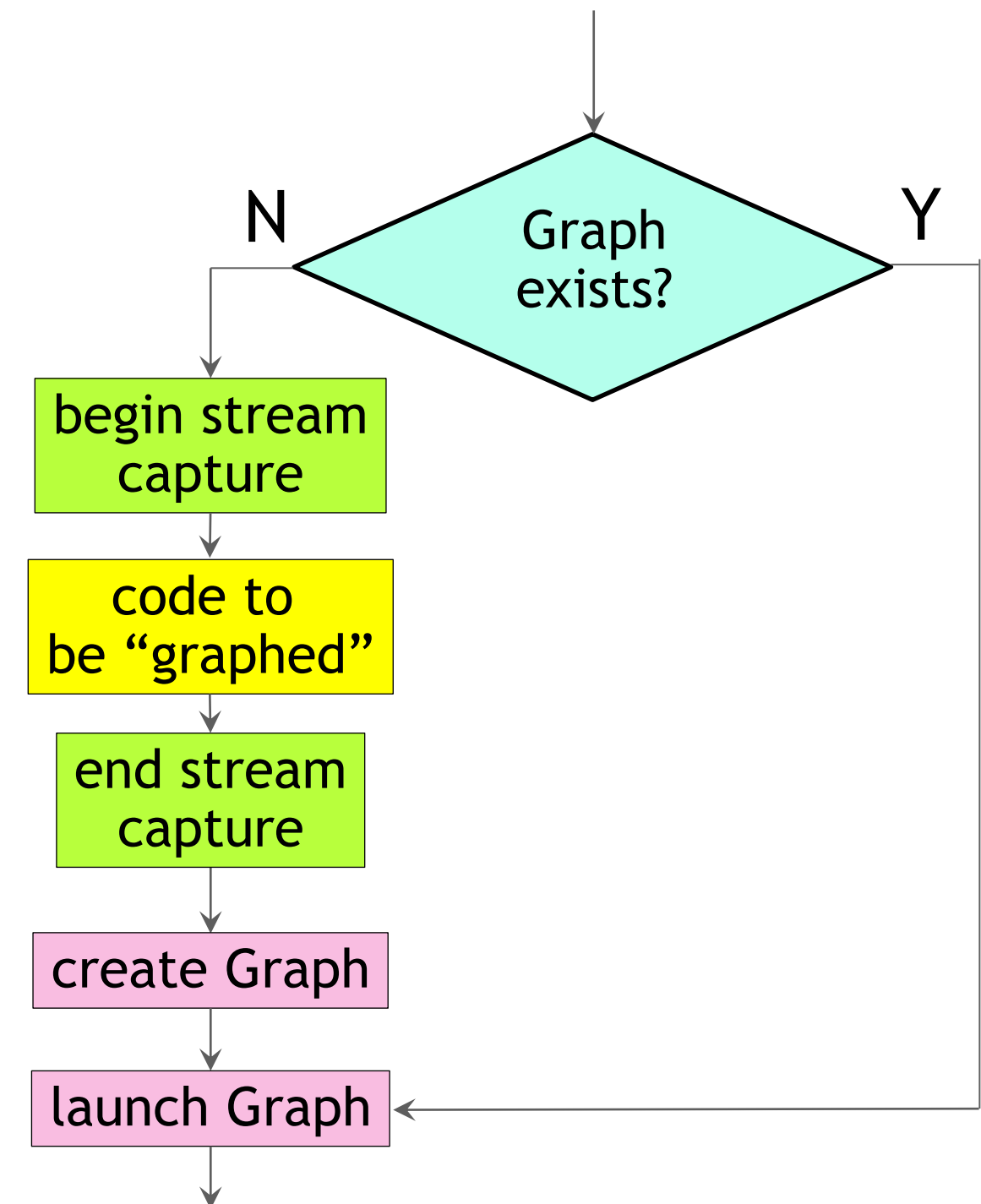
WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

- ▶ Pros: convenient, done in situ
- ▶ Cons: somewhat less control



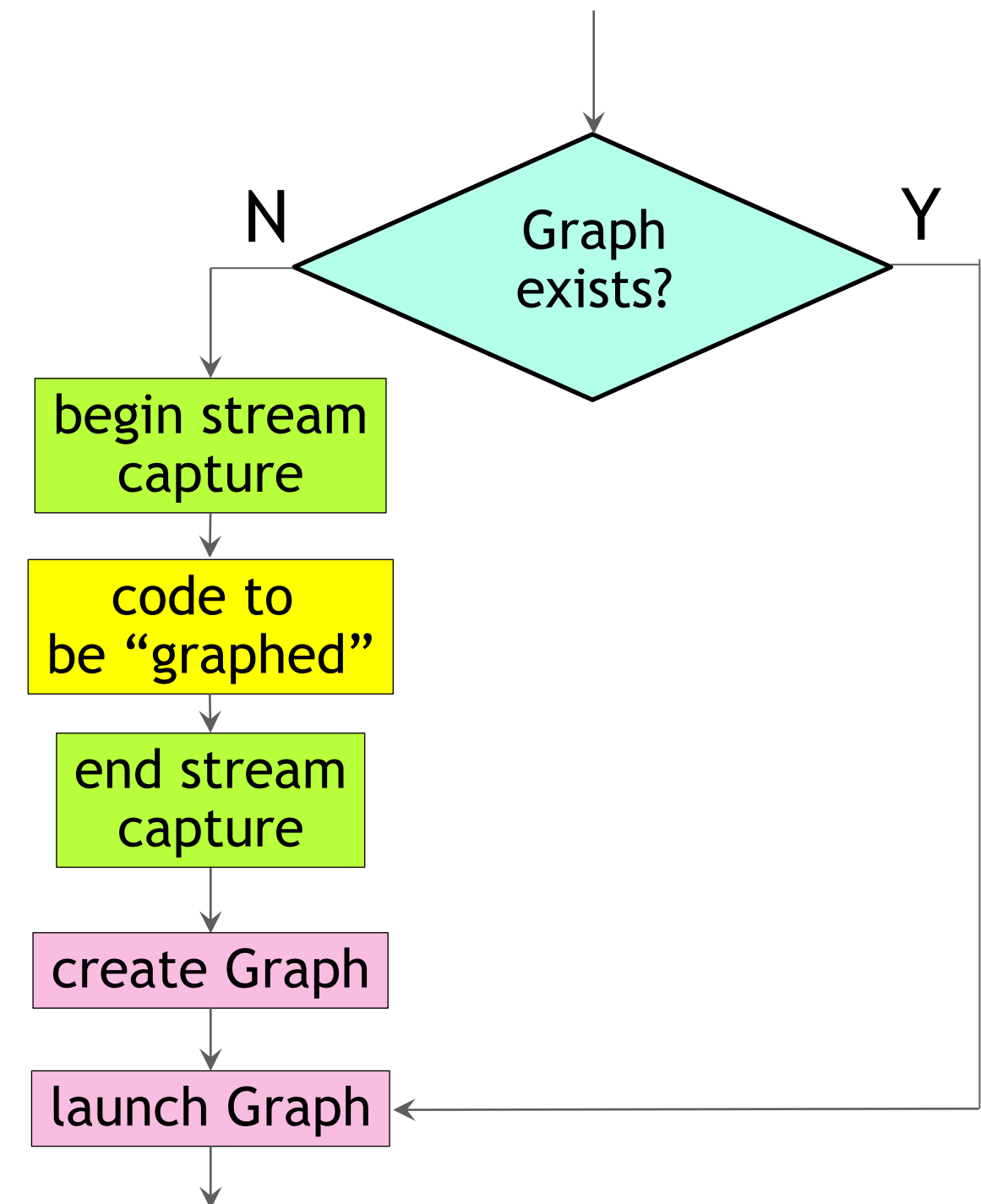
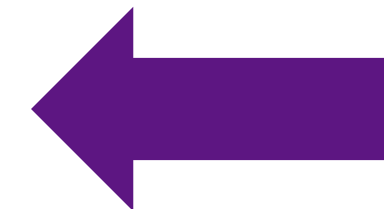
WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

- ▶ Pros: convenient, done in situ
- ▶ Cons: somewhat less control



APPLICATION EXAMPLE

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
tight_loop(); //function containing many small kernels
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance;
```

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);  
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance;
```

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);  
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

```
cudaGraphLaunch(instance, stream);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance; bool capture = false;
...
if (!capture)
{
    // we haven't captured the graph yet
    cudaGraph_t graph;
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(); //function containing many small kernels
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    capture = true;
}
cudaGraphLaunch(instance, stream);
```


REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

Great candidate for CUDA Graph, except:

- ▶ **parameters** and **topology** (number of kernels) may change
- ▶ In principle, need to check all
- ▶ Graphs only give benefit if they can be relaunched

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

printf is your friend

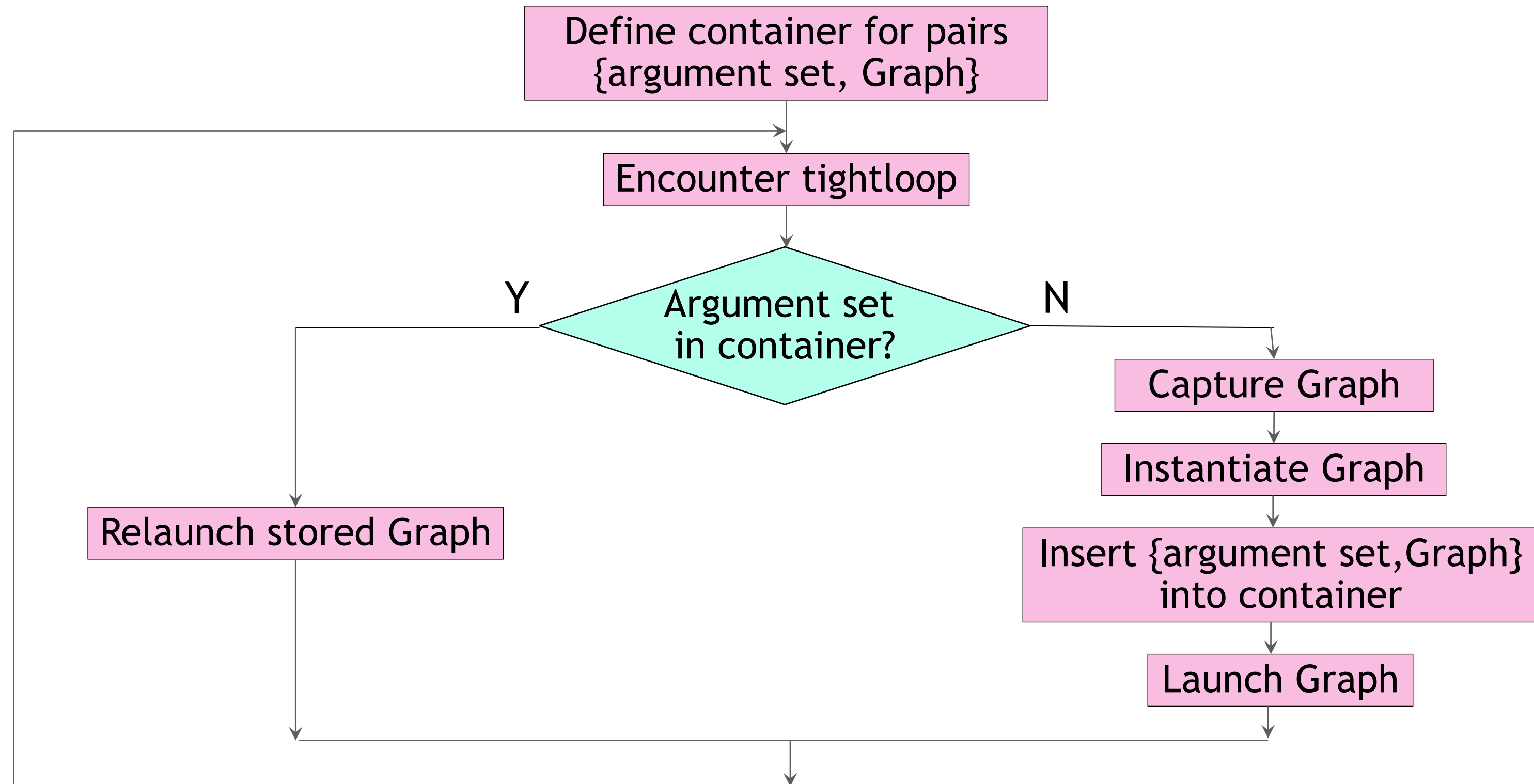
Great candidate for CUDA Graph, except:

- ▶ **parameters** and **topology** (number of kernels) may change
- ▶ In principle, need to check all
- ▶ Graphs only give benefit if they can be relaunched

Workload size	Small	Large
#tightloop calls	70	1540
#iterations	252	1260
Unique argument sets	5	3

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?



SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
```

```
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
```

```
...
```

```
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
```

```
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
```

```
cudaStreamEndCapture(stream, &graph);
```

```
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

```
cudaGraphLaunch(instance, stream);
```

```
}
```

Introduce container
plus elements

Add this if arguments
do change

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);

    cudaGraphLaunch(instance, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    // add this graph to the map of saved graphs
    MapOfGraphs.insert(make_pair(sextet, instance));
    cudaGraphLaunch(instance, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Add Graph to container

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    // add this graph to the map of saved graphs
    MapOfGraphs.insert(make_pair(sextet, instance));
    cudaGraphLaunch(instance, stream);
}
else
{ // relaunch previously captured graph
    cudaGraphLaunch(it->second, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Add Graph to container

Launch Graph in
container

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
  cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
  tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
  cudaStreamEndCapture(stream, &graph);
  cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
  // add this graph to the map of saved graphs
  MapOfGraphs.insert(make_pair(sextet, instance));
  cudaGraphLaunch(instance, stream);
}
else
{ // relaunch previously captured graph
  cudaGraphLaunch(it->second, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Plus cmpMarker function to
compare different instances
of grid_dim, block_dim,
first_step, delta, stream,
params.X

Add Graph to container

Launch Graph in
container

CONCERNS

Recognizing and saving CUDA Graphs requires app knowledge

Need to write custom comparison function

Tempting to create shortcuts: “I know only *first_step* and *stream* may change, so I only check those”

Easy to overlook potential show stoppers: “I didn’t realize *first_step* changes Graph topology”

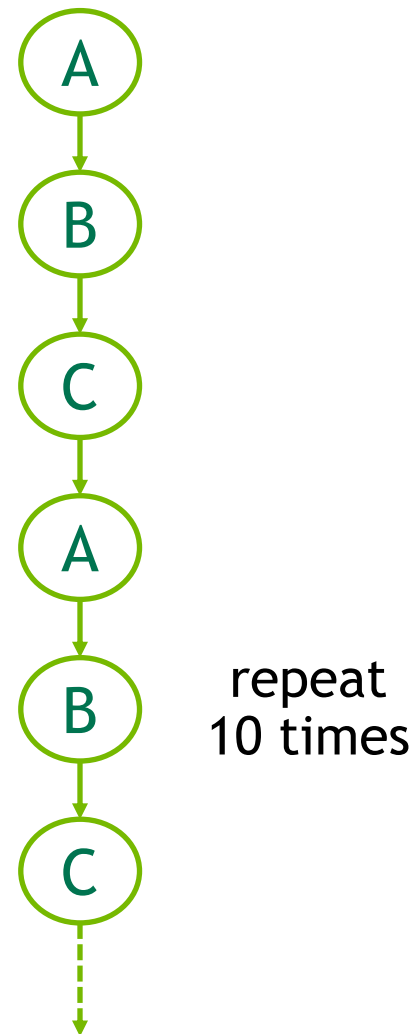
Hard to maintain:

- ▶ What if fields get added to struct params?
- ▶ What if argument changes depend on workload?

SOLUTION 2: UPDATE GRAPHS

Kernel and Graph execution restrictions

Stream Launch



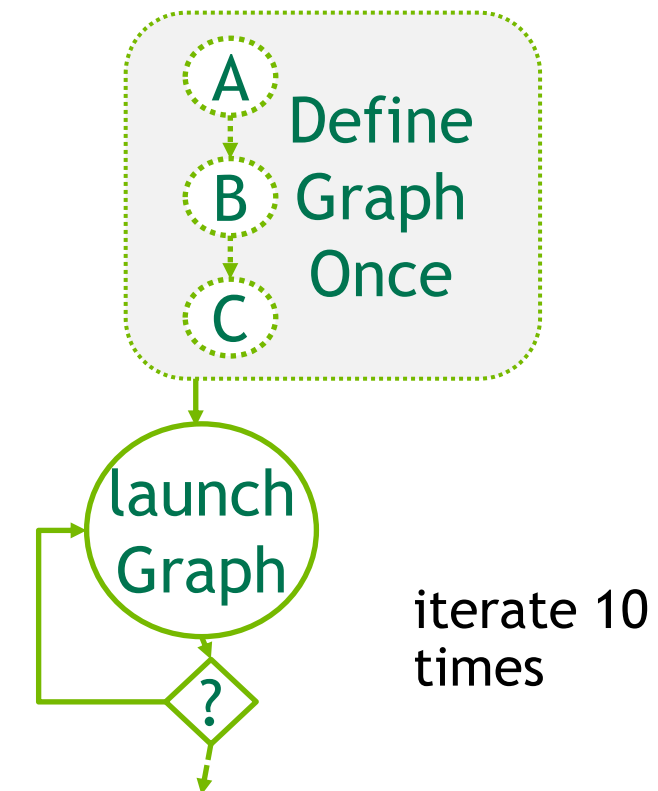
Parameters: **may** change
Topology: **may** change

Graph Update

- Compare existing Graph with newly captured Graph
- If possible, update parameters of existing Graph

Parameters: **may** change
Topology: **may not** change

Graph Relaunch

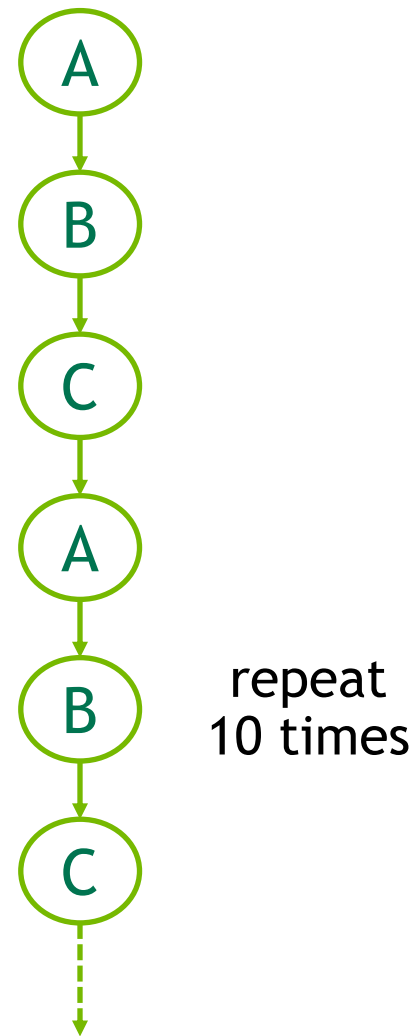


Parameters: **may not** change
Topology: **may not** change

SOLUTION 2: UPDATE GRAPHS

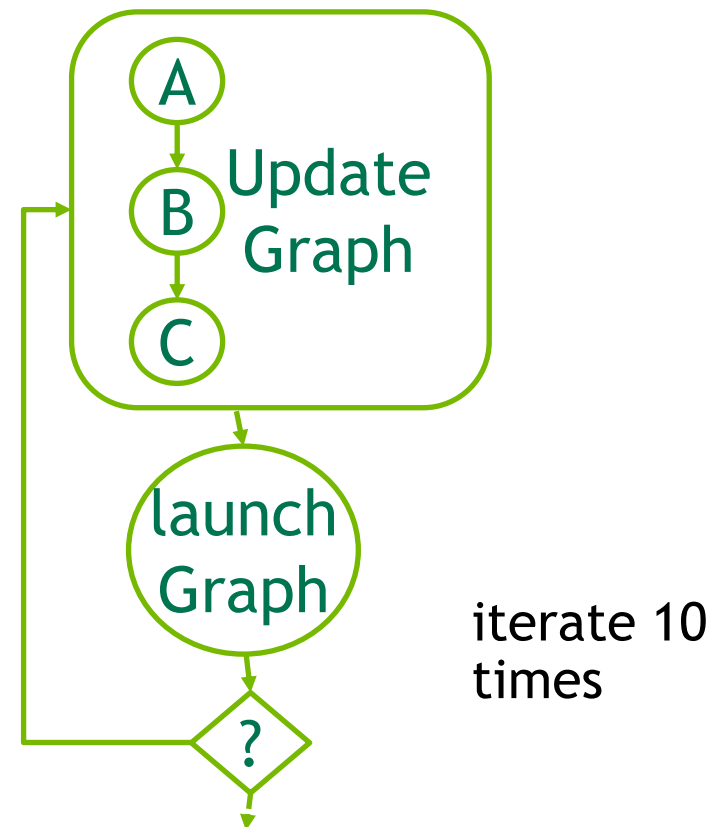
Kernel and Graph execution restrictions

Stream Launch



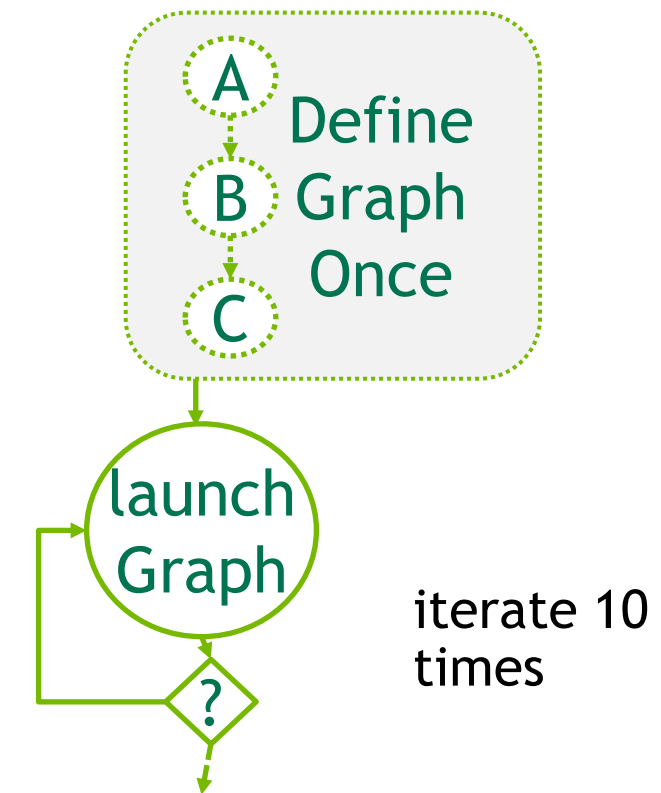
Parameters: **may** change
Topology: **may** change

Graph Update



Parameters: **may** change
Topology: **may not** change

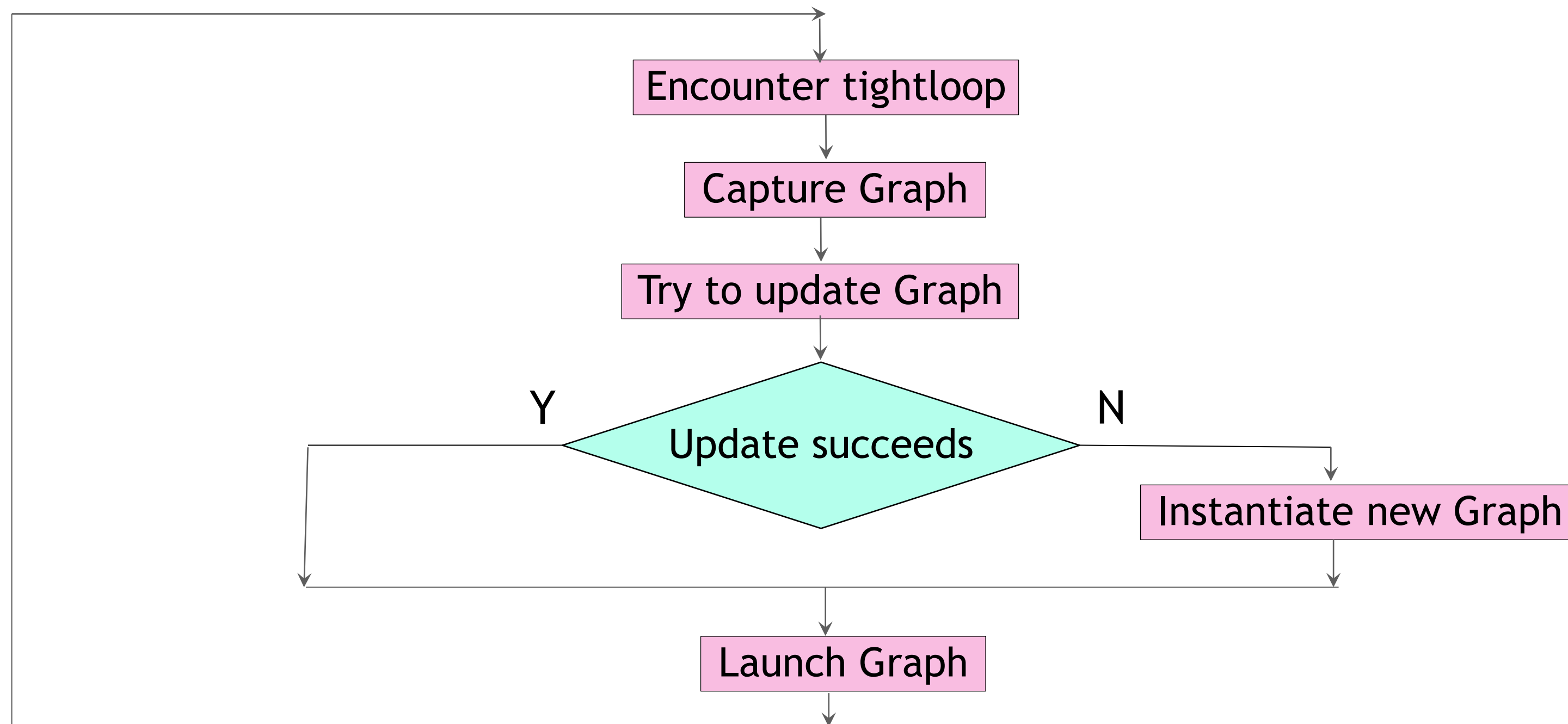
Graph Relaunch



Parameters: **may not** change
Topology: **may not** change

SOLUTION 2: UPDATE GRAPHS

No STL skills required



SOLUTION 2: UPDATE GRAPHS

```
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);  
cudaStreamEndCapture(stream, &graph);
```

ALWAYS capture Graph

SOLUTION 2: UPDATE GRAPHS

```
bool have_exec = false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!have_exec) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    have_exec = true;
}

cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

SOLUTION 2: UPDATE GRAPHS

```
bool have_exec = false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!have_exec) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    have_exec = true;
} else {
    // graph already exists; try to apply changes
    if (cudaGraphExecUpdate(instance, graph, NULL, &update)

}

cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

Update Graph and go

SOLUTION 2: UPDATE GRAPHS

```
bool have_exec = false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!have_exec) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    have_exec = true;
} else {
    // graph already exists; try to apply changes
    if (cudaGraphExecUpdate(instance, graph, NULL, &update) != cudaSuccess)
        cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
}
cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

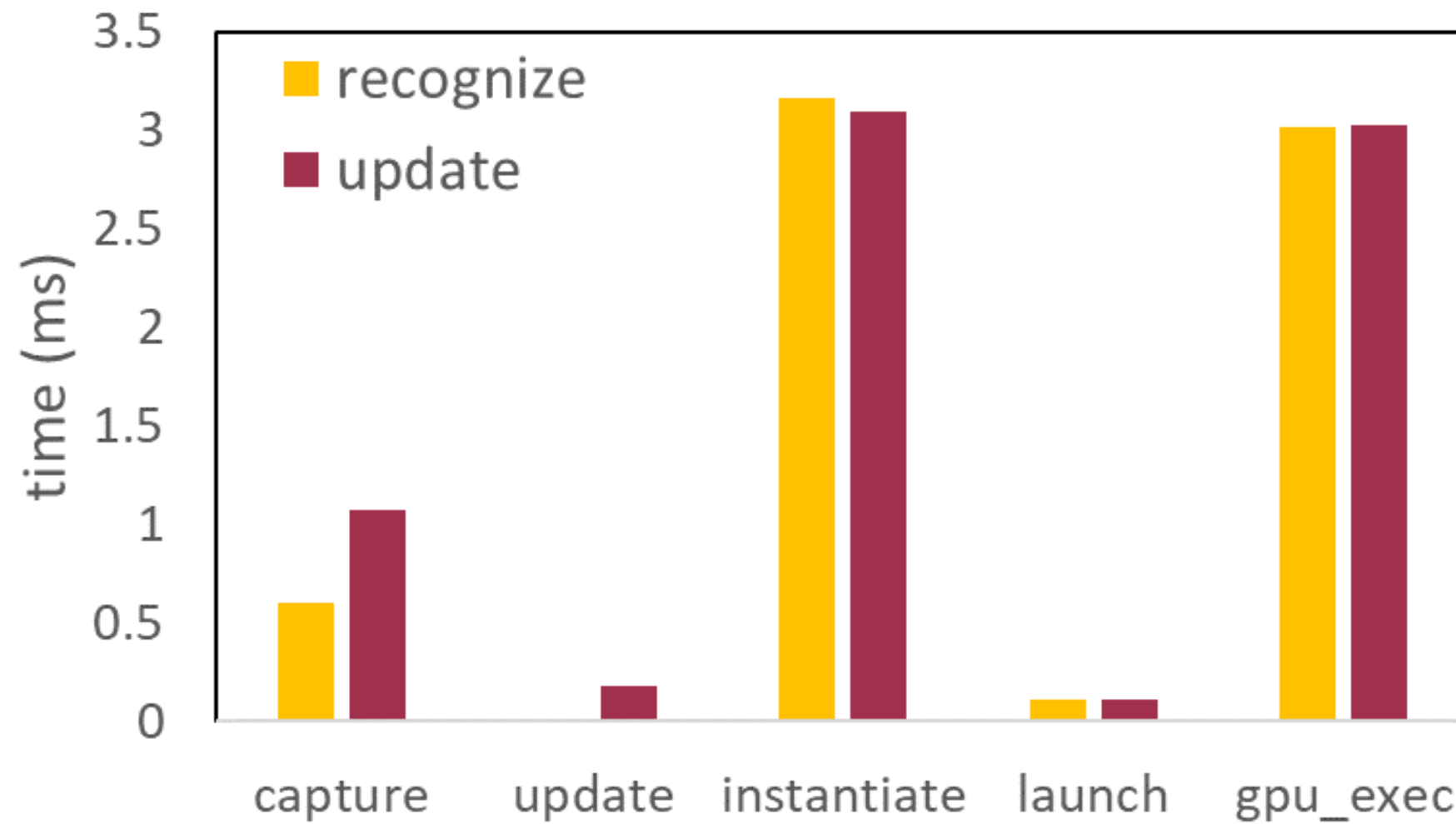
Update Graph and go

Update failed, reinstantiate

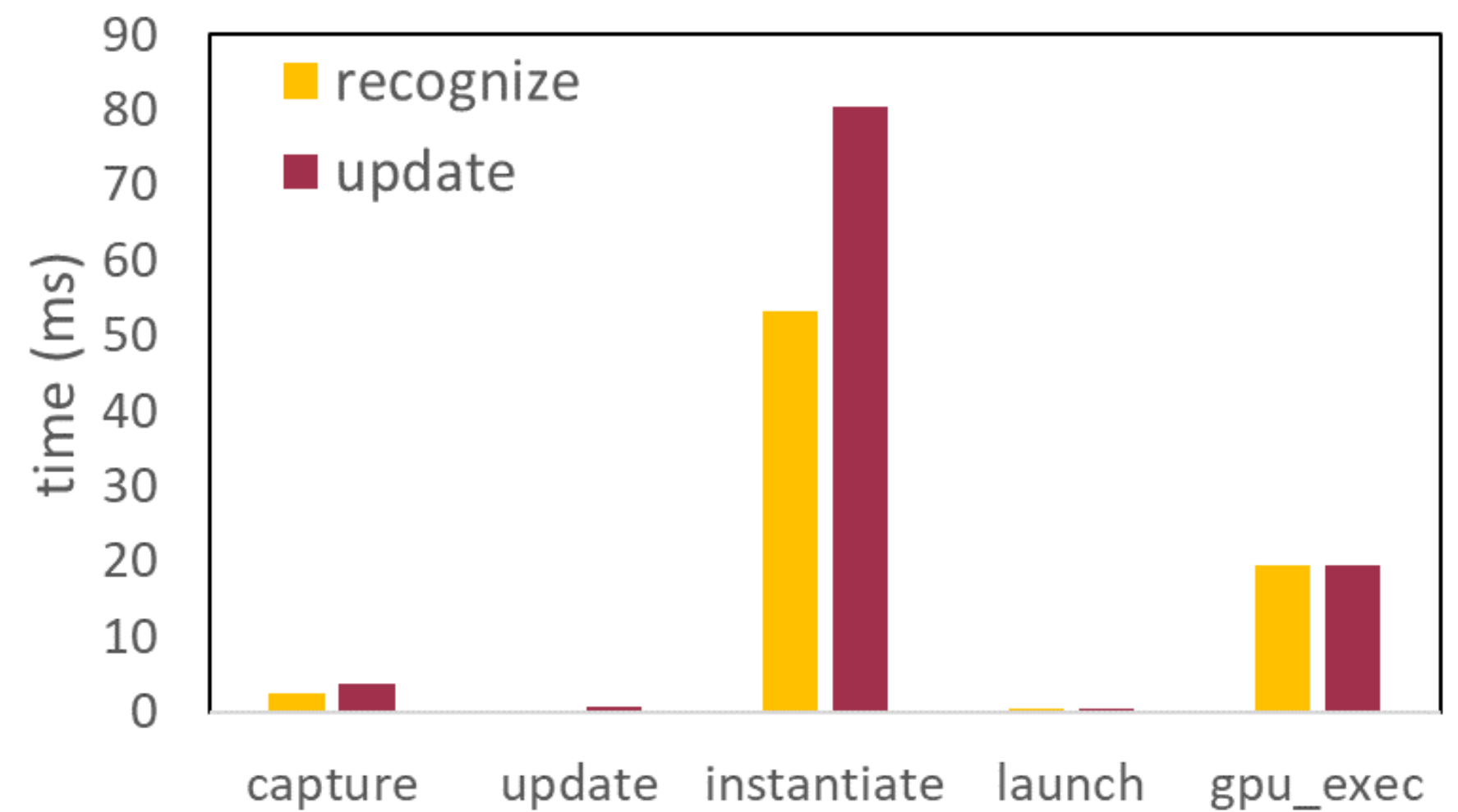
SOLUTION 2: UPDATE GRAPHS

Capture, update, launch cheap compared to instantiate, run

Average time per operation, small workload



Average time per operation, large workload



SOLUTION 2: UPDATE GRAPHS

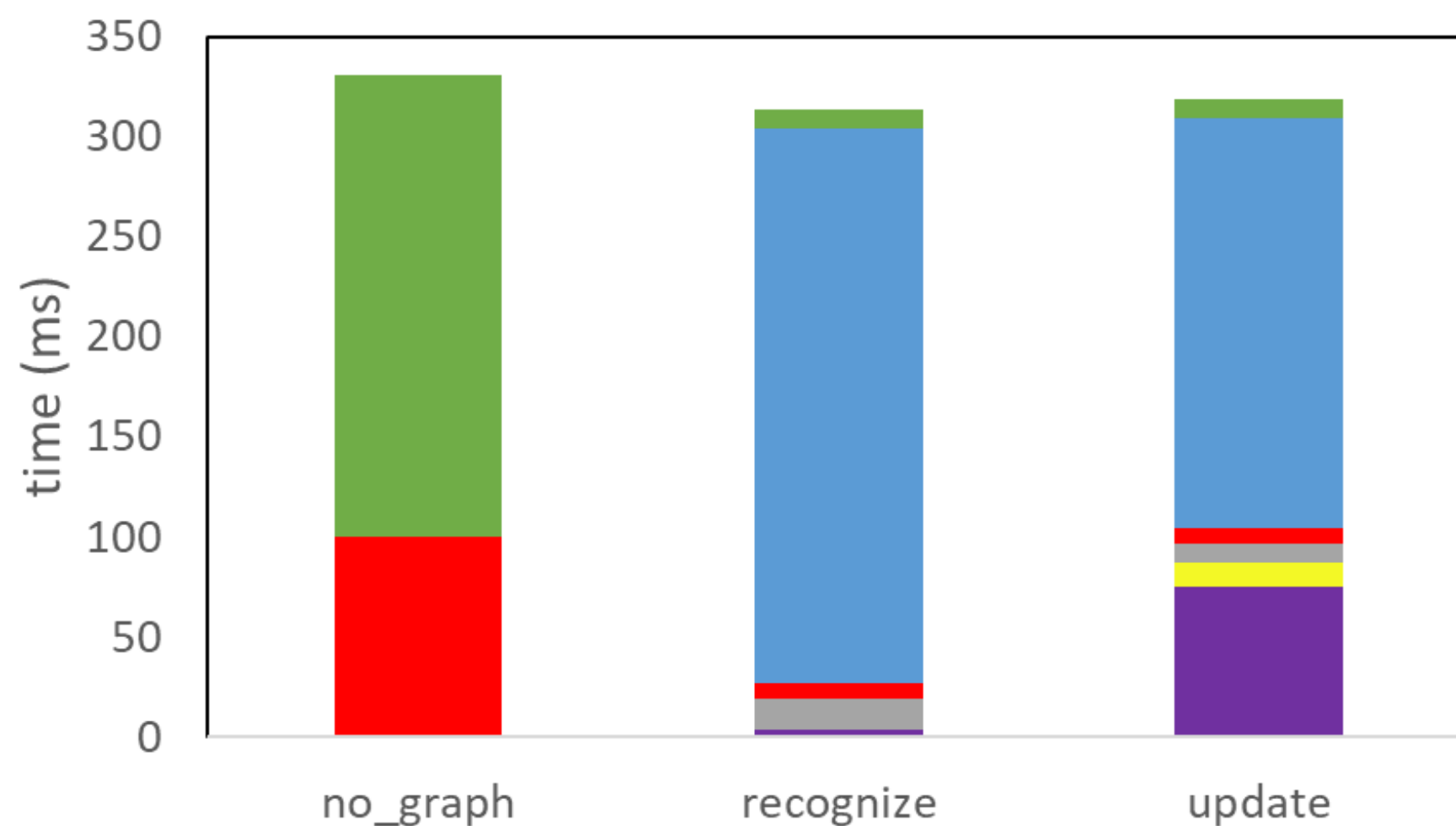
Expensive instantiations are infrequent

Workload size	Small		Large	
Kernels/Graph	504		2520	
Graph launches	70		1540	
Method	update	recognize	update	recognize
Captures	70	5	1540	3
Updates	69	---	1539	---
Instantiations	3	5	1	3

COMPARING RECOGNIZE AND UPDATE

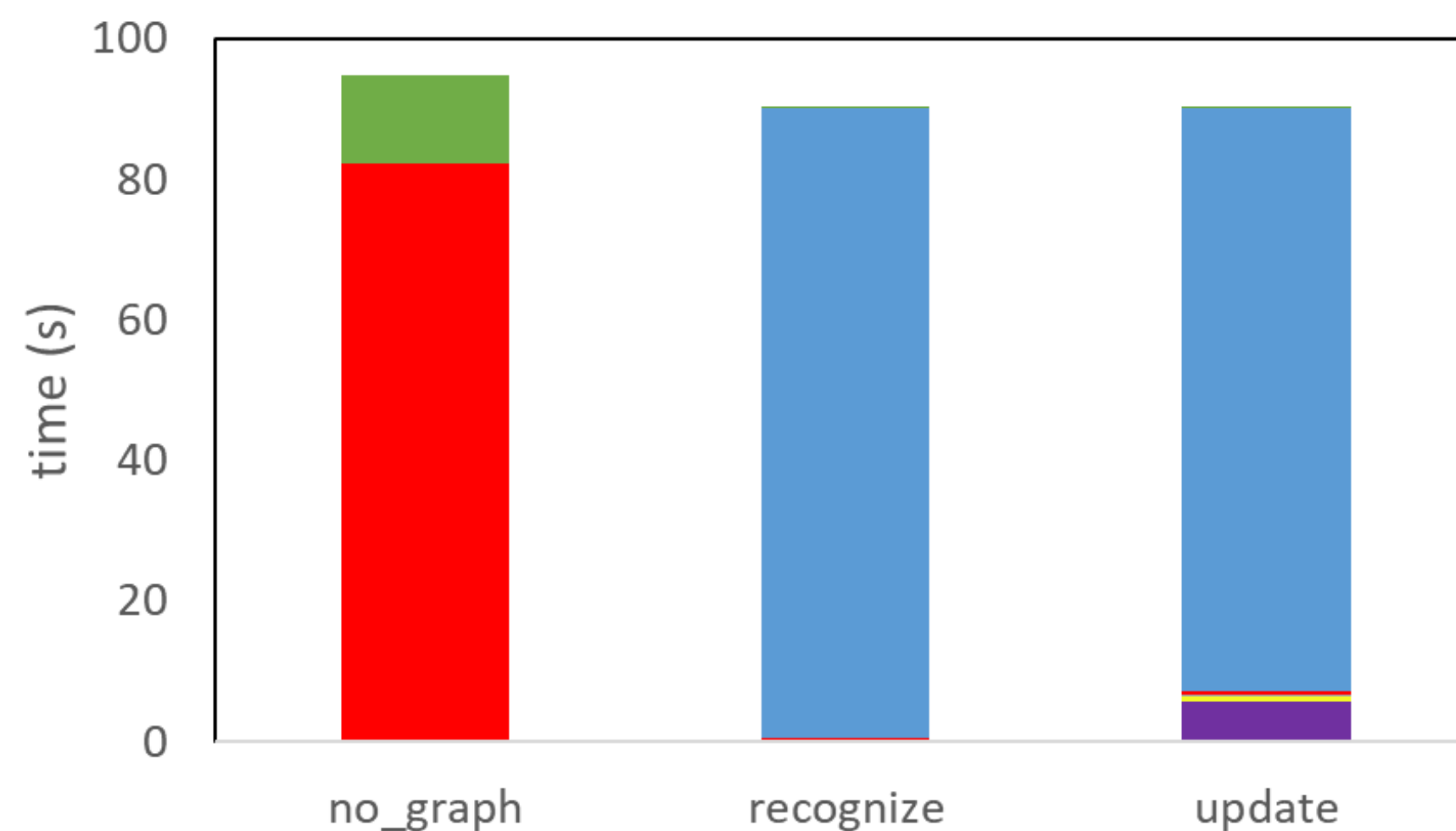
Launch time reduces greatly with CUDA Graphs

Average time per graphable region, small workload



capture update instantiate launch synch other

Average time per graphable region, large workload



capture update instantiate launch synch other

“EFFORTFUL” CUDA GRAPH UPDATE

Partial Graph update may accelerate code

If topology does not change AND you know exact node changes:

- ▶ No need to recapture & update whole CUDA Graph
- ▶ Can use CUDA Graph update APIs to modify individual Graph node(s), e.g.:
`cudaGraphExecKernelNodeSetParams()`

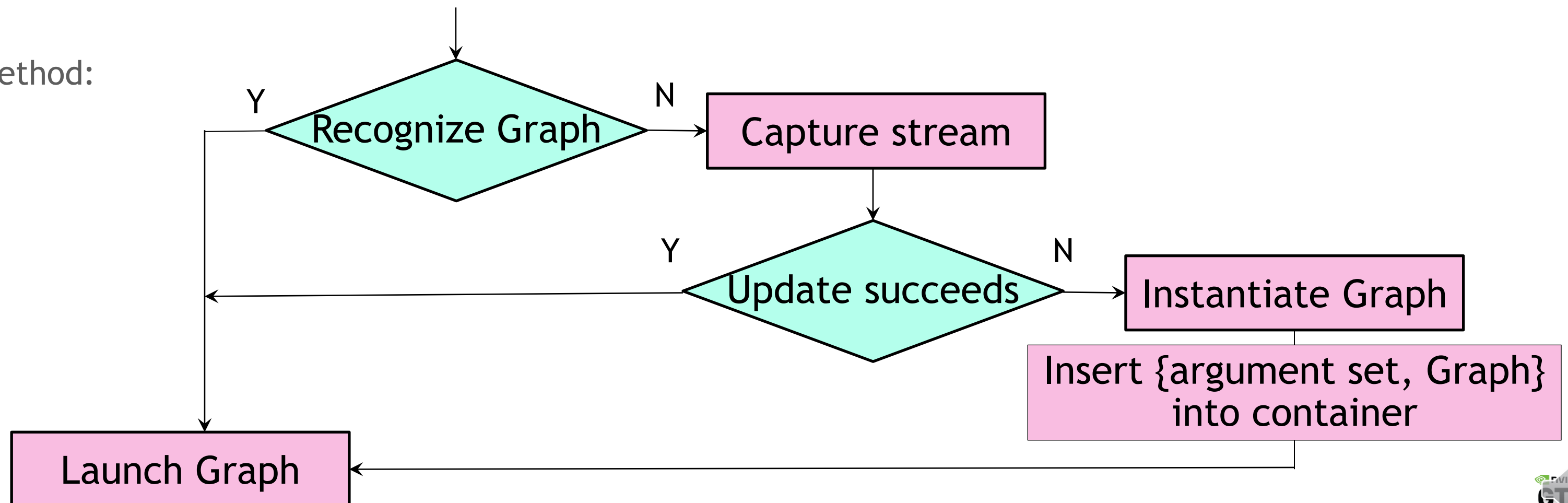
“EFFORTFUL” CUDA GRAPH UPDATE

Partial Graph update may accelerate code

If topology does not change AND you know exact node changes:

- ▶ No need to recapture & update whole CUDA Graph
- ▶ Can use CUDA Graph update APIs to modify individual Graph node(s), e.g.:
`cudaGraphExecKernelNodeSetParams()`

Consider hybrid method:



COMPARING RECOGNIZE AND UPDATE

Recognize

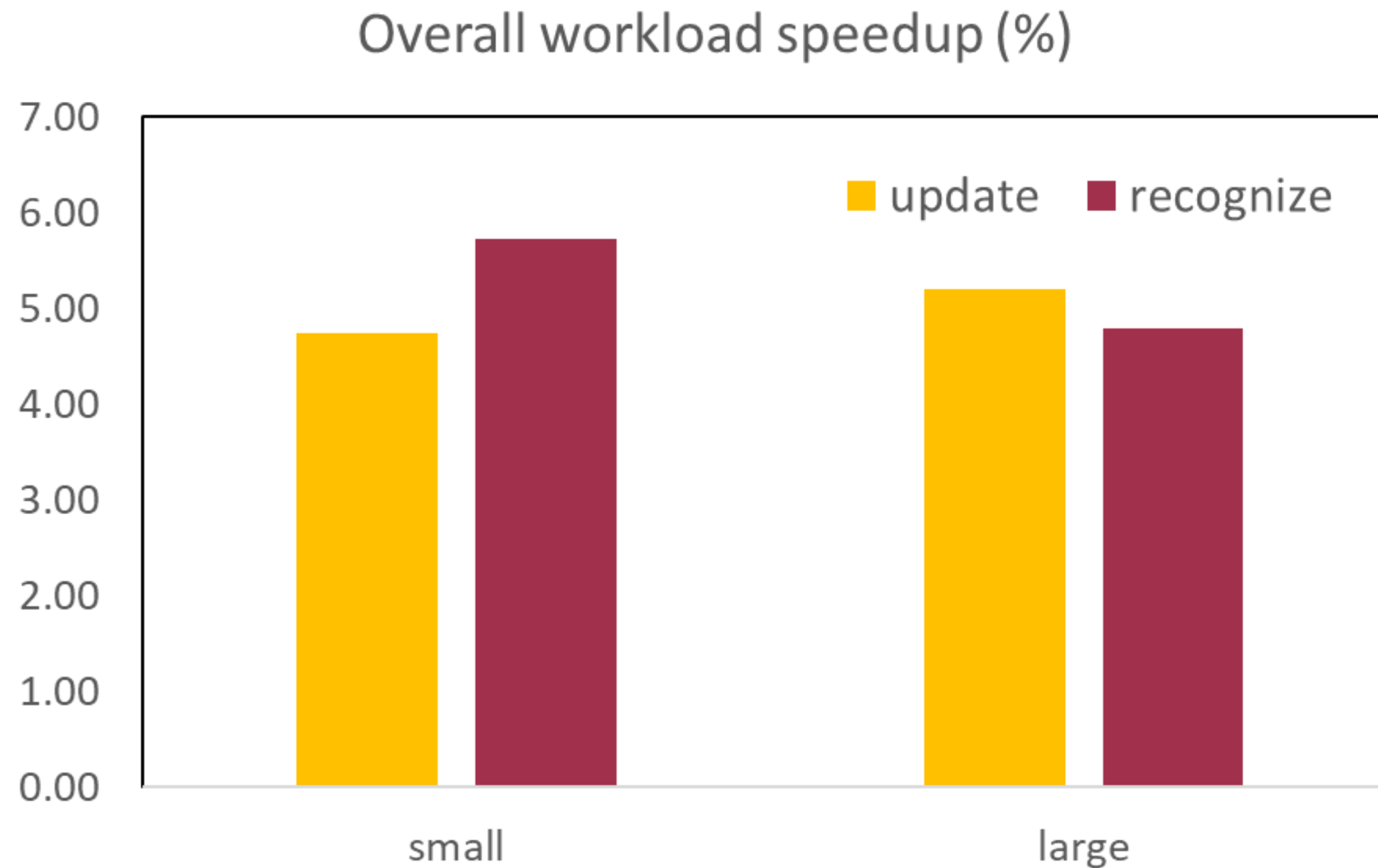
- ▶ Best when small number of unique Graphs, repeated many times
- ▶ Independent of order in which graphs encountered
- ▶ (Almost) no overhead when saved Graph is recognized
- ▶ May have to save many Graphs

Update

- ▶ Best when small number of unique topologies, repeated many times
- ▶ Worst when cycling between Graphs with different topologies
- ▶ Always some stream capture overhead
- ▶ Never need to save more than one Graph

COMPARING RECOGNIZE AND UPDATE

Graphable region less than 30%/20% of overall small/large workload



SUMMARY

CUDA Graphs can give substantial performance benefits with little effort

Criteria for introducing CUDA Graphs:

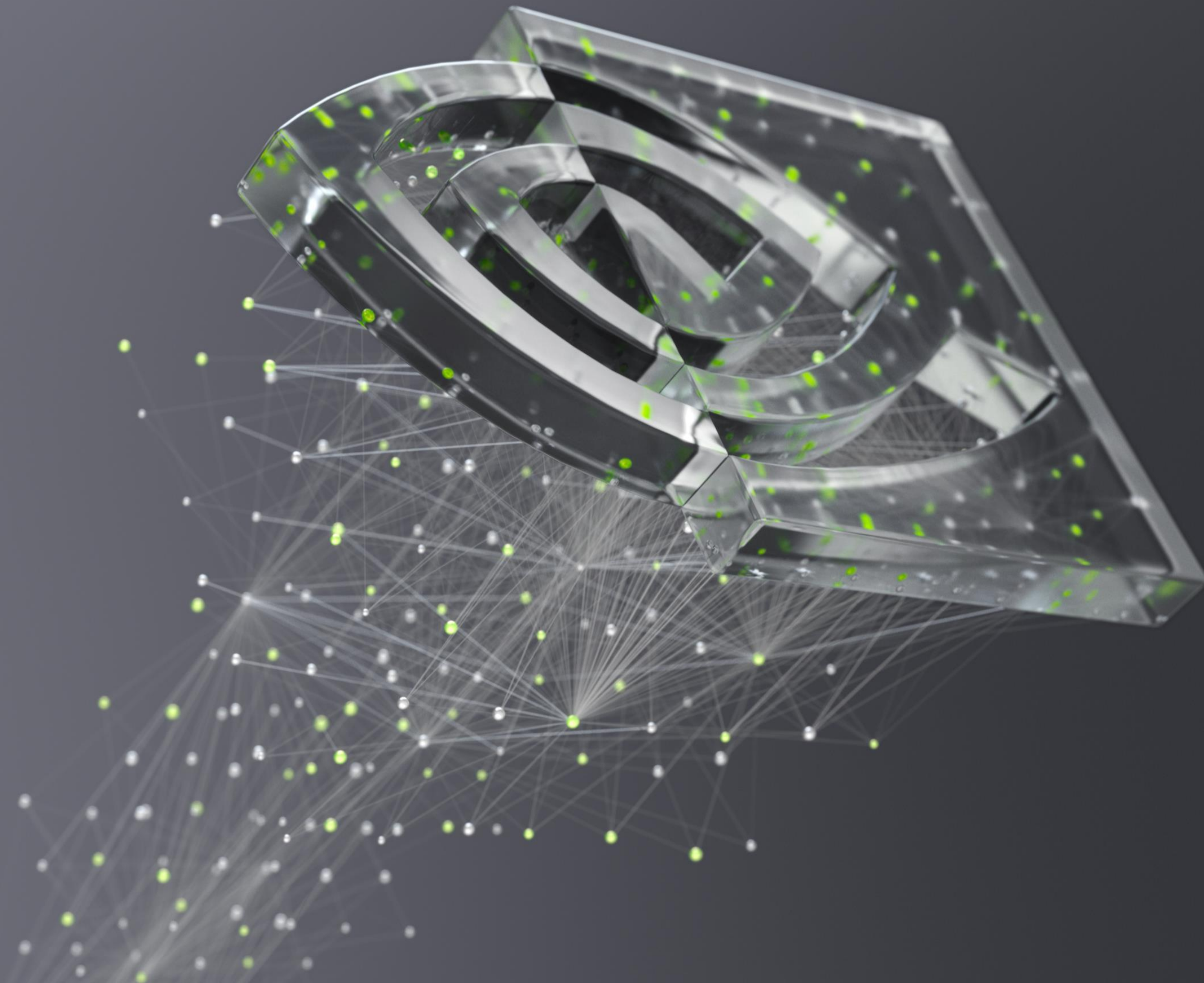
- ▶ Repeated pattern of set of small kernels
- ▶ Only asynchronous operations

Efficient way to create CUDA Graphs

- ▶ Stream capture

Different approaches for different situations

- ▶ If identical sets of kernels recur: almost no work
- ▶ If topology and/or parameters change frequently: consider saving and recognizing Graphs
- ▶ If topology changes infrequently: consider updating Graphs



 NVIDIA
GTC